COURT IQ - CC 302 CODE REVIEWER

Study guide for your final project defense | Flutter + Dart | Basketball training app

> **Your 20-second summary:** Court IQ is an offline-first basketball training companion. Users browse drills, use a timer, save sessions, review progress, and manage custom drills. It uses Flutter widgets for the UI, `ChangeNotifier` and `setState` for state, `shared_preferences` for local persistence, and HTTP/JSON for an ESPN NBA headlines feed. The five-tab floating navigation is a reusable custom widget.

1. What problem does our app solve?

Problem: Student basketball players practice without a plan or a simple record of progress.

Solution: Court IQ puts practice drills, timer controls, persistent workout history, weekly goal tracking and basketball news in one mobile-friendly UI.

Target users: Student-athletes, beginner and intermediate basketball players, and people practicing independently.

Don't overclaim: This is not an AI coach, social network, cloud database, workout verifier or login system. The Community screen is a basketball-news feed, not live user-to-user chat.

2. How the app is organized (architecture)

```
main.dart  ->  CourtIqApp (MaterialApp, overall theme)
               |
               +-- MainShell (StatefulWidget, currentTab)
                    |
                    +-- FloatingNav (5 tab buttons)
                    +-- HomeScreen      -> WorkoutScreen
                    +-- TrainScreen     -> WorkoutScreen / _DrillFormSheet
                    +-- ProgressScreen  -> training history / delete
                    +-- CommunityScreen -> NbaService -> ESPN API
                    +-- ProfileScreen   -> _NameDialog / weekly goal
                          |
AppStore (ChangeNotifier) -> shared_preferences on device
                         |
TrainingRecord and Drill models -> JSON maps
```

Key idea: Screens display data and accept taps. `AppStore` keeps shared app data; models describe the data; `NbaService` handles the online request. Separating these makes the code easier to test and explain.

3. File-by-file reviewer

File	What it does	Main code concept
`lib/main.dart`	Starts the app, loads saved data, provides `MaterialApp`, displays five-tab shell.	`main()`, `StatelessWidget`, `StatefulWidget`, `setState`, `IndexedStack`, `AnimatedBuilder`, `AppBar`
`lib/theme/app_theme.dart`	Holds reusable colors and application-wide `ThemeData`.	Constants, color scheme, consistent visual styling
`lib/models/training.dart`	Defines `Drill` and `TrainingRecord`; built-in practice library; JSON conversion.	Classes, constructors, `final`, lists, factories, `toJson` / `fromJson`
`lib/services/app_store.dart`	Saves workouts, custom drills, user name and goal; notifies UI changes.	`ChangeNotifier`, `notifyListeners`, `Future`, `shared_preferences`, `jsonEncode`
`lib/services/nba_service.dart`	Fetches and parses NBA headlines, handles HTTP and malformed data.	`http.Client.get`, `await`, `jsonDecode`, exceptions, 12-second timeout

File	What it does	Main code concept
`lib/screens/home_screen.dart`	Personalized hero, saved totals, goal progress and workout launch.	`StatelessWidget`, `ListView`, callbacks, image assets
`lib/screens/train_screen.dart`	Searches/filters drills and opens the custom drill add/edit form.	`StatefulWidget`, `setState`, `ListView.builder`, `Form`, `TextFormField`
`lib/screens/workout_screen.dart`	Countdown, instructions, pause/reset and manually save a session.	`Timer.periodic`, lifecycle observer, `Navigator`, `async/await`
`lib/screens/progress_screen.dart`	Calculates 7-day chart and shows/delete saved sessions.	`ListView.builder`, date calculations, delete confirmation
`lib/screens/community_screen.dart`	Renders live headlines, loading, error/retry, empty and sample states.	`StatefulWidget`, API service, `RefreshIndicator`, conditional UI
`lib/screens/profile_screen.dart`	Edits name and goal; clears local records with confirmation.	Dialog, state, validation, persistent settings
`lib/widgets/floating_nav.dart`	The floating pill ribbon with five icons and labels.	Custom reusable `StatelessWidget`, callbacks, `BackdropFilter`, accessibility semantics
`lib/widgets/ui.dart`	Reusable cards, action buttons, headers, empty-state and delete dialog.	Component reuse, HCI consistency
`test/*.dart`	Checks persistence, deletion, navigation and mocked API responses.	Unit tests, widget tests, `MockClient`

4. Core Dart and Flutter concepts with examples

A. `main()` and `runApp()`

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final store = AppStore();
  await store.load();
  runApp(CourtIqApp(store: store));
}
```

Explain: `main()` is the entry point. `ensureInitialized()` prepares Flutter's binding for plugin use before building widgets. `await store.load()` loads saved preferences before the first UI frame. `runApp()` starts the widget tree.

B. `StatelessWidget` vs `StatefulWidget`

- `StatelessWidget`: UI with no private, changing screen state. Example: `HomeScreen`, which reads the shared `AppStore` supplied by its parent.
- `StatefulWidget`: UI with state that changes while the screen exists. Example: `TrainScreen` (search/filter), `MainShell` (selected tab), `WorkoutScreen` (seconds and running status).
- **Subtle point:** A `StatelessWidget` **can still update its appearance** if a listening parent rebuilds with updated data. "Stateless" means it does not own mutable local `State`, not that it can never rebuild.

C. `setState()` vs `notifyListeners()`

```
setState(() => currentTab = index); // only state belonging to this widget
notifyListeners();                  // shared AppStore changed
```

`setState` tells Flutter to rebuild that widget's subtree. `notifyListeners()` tells listeners of `AppStore` that shared data changed. `AnimatedBuilder(animation: widget.store, ...)` rebuilds the shell when workouts/name/goal change.

D. Custom widgets

`SurfaceCard`, `PrimaryAction`, `Kicker`, `EmptyPanel`, `SectionHeader`, and `FloatingNav` are reused to avoid copying layout and styling everywhere. This supports **consistency** and maintainability.

E. `ListView.builder`

```
ListView.builder(
  itemCount: drills.length,
  itemBuilder: (context, index) {
    final drill = drills[index];
    return _DrillCard(drill: drill, onTap: () => _openDrill(drill));
  },
)
```

`itemBuilder` creates rows as needed rather than making all rows up front, which helps efficiently display long lists. In Court IQ it's used in Train, Progress and Community.

F. Dart classes, `final`, `const`, constructors and getters

A `Drill` is a model with fields such as `id`, `title`, `category`, `minutes` and `steps`. `final` means the field reference cannot be reassigned after construction. `const` creates a compile-time constant value if all inputs are constant. A getter like `totalSessions => records.length` calculates a value without a function-call syntax.

G. `Future`, `async` and `await`

A `Future` represents a result that might arrive later: reading storage, saving preferences or requesting data online. `await` lets code wait without blocking the UI thread. `try/catch` handles errors; `if (mounted)` or `context.mounted` avoids updating a widget after it has been removed.

H. `Navigator.push` and `Navigator.pop`

`Navigator.push` opens a new screen such as `WorkoutScreen`; `Navigator.pop(context, true)` returns to the previous screen and sends back a result. The floating bar changes `currentTab` in `MainShell` instead of pushing five unrelated routes.

I. Forms and validation

The custom-drill bottom sheet uses `GlobalKey<FormState>`, `TextFormField` validators, and `DropdownButtonFormField`. Title and description cannot be empty; duration must be between **1 and 180 minutes**. Controllers belong to the sheet widget and are disposed when the widget is removed.

J. Human-Computer Interaction (HCI)

- **Consistency:** charcoal/orange palette, shared widgets, same layout patterns.
- **Visibility:** five labeled tabs; counts and progress indicators.
- **Feedback:** SnackBars when actions finish or fail; spinner for API loading.
- **User control:** Cancel options and deletion confirmation.
- **Error prevention:** required-form validation, duration limits, goal limits (1–7).
- **Recovery:** retry button for API errors and clearly labeled offline tips.
- **Accessibility:** semantics on navigation items, tooltips, scrollable screens and contrasting text; check text sizes on your emulator.
- **Responsive design:** `Expanded`, `SafeArea`, `ListView`, `FittedBox` and padding help fit mobile screens.

5. How to explain the major features in code

Feature 1 - Switching tabs

1. `FloatingNav` draws five choices: Home, Train, Progress, Community, Profile.
2. When tapped, `onSelected(index)` calls `switchTab(index)` in `MainShell`.
3. `setState` changes `currentTab`.
4. `IndexedStack(index: currentTab, children: pages)` shows the selected screen while preserving tab state.

Instructor question: Why `IndexedStack`? **Answer:** It switches visible content without destroying state every time you change tabs; for example, the Train search/filter can remain available when you return.

Feature 2 - Logging a workout

1. User selects a `Drill` in Home or Train.
2. A `WorkoutScreen` opens via `Navigator.push`.
3. The timer supports start/pause/reset. A real-time end timestamp helps correct the countdown after resuming the app.
4. The user taps **Complete & save workout**.
5. `AppStore.addSession(drill)` adds a `TrainingRecord` with title, category, duration, date and unique ID.
6. `_saveRecords()` serializes the list using `jsonEncode` and stores the string locally.
7. `notifyListeners()` updates Home, Progress and Profile counts.

Be precise: It logs *the user-reported completion*. It does not sense whether the player physically completed the exercises.

Feature 3 - Create, edit and delete a drill (CRUD)

- **Create:** Form validates and creates a `Drill` with `isCustom: true`.
- **Read:** Train displays `allDrills`, combining `defaultDrills` and `customDrills`.
- **Update:** `saveCustomDrill` finds the matching `id` and replaces that entry.
- **Delete:** `confirmDelete` asks first; then `deleteCustomDrill` removes the entry and persists the result.
- Built-in drills are **read-only**, so only user-created drills show edit/delete buttons.

Feature 4 - API integration

```
final response = await _client.get(endpoint, headers: {
  'Accept': 'application/json',
}).timeout(const Duration(seconds: 12));
if (response.statusCode != 200) throw Exception('HTTP error');
final decoded = jsonDecode(response.body);
```

The service requests JSON from an ESPN NBA-news endpoint, checks HTTP status and JSON structure, and maps article fields into the `NewsStory` model. The Community screen displays a spinner while loading, articles on success, an empty state if no articles, and retry/offline tips on failure. **Offline samples are labeled and are not API results.**

Live endpoint: `https://site.api.espn.com/apis/site/v2/sports/basketball/nba/news?limit=12`

Possible errors: no internet, DNS failure, API server error, non-200 response, JSON format change, timeout. The request isn't used to upload or store workouts.

Feature 5 - Local persistence

`SharedPreferences` stores simple device-local key/value pairs. For lists, we first convert models to JSON maps (`toJson`), encode them as strings (`jsonEncode`), and decode/reconstruct them (`jsonDecode` and `fromJson`) at launch.

Why not a database? For a small student project and limited local data, `shared_preferences` is simple and sufficient. A larger production app would normally use a database or backend for more complex data and accounts.

Feature 6 - Delete confirmation

1. User taps red trash icon beside a saved workout or custom drill.
2. `showDialog<bool>` opens Cancel/Delete choices.
3. If cancelled, it returns `false` and nothing changes.
4. If confirmed, `removeWhere` removes the item with the matching unique `id`.
5. JSON is saved again; `notifyListeners()` updates the UI and a SnackBar confirms deletion.

Deleting a completed record is **not** the same as deleting a drill template. Explain which list changes (`records` versus `customDrills`).

6. Instructor questions with short model answers

1. Why did you choose Court IQ? To help players practice with a structured drill list and monitor completed sessions.

2. Why Flutter and Dart? Flutter builds mobile UIs from reusable widgets; Dart supports object-oriented models and asynchronous APIs.

3. What is a widget? The basic building block of a Flutter interface; screens, buttons, text and layouts are widgets.

4. Where does execution start? In `main()` in `lib/main.dart`.

5. Why call `WidgetsFlutterBinding.ensureInitialized()`? To prepare Flutter/plugin services before `runApp`, since preferences load first.

6. Why make `MainShell` Stateful? It owns `currentTab`, which changes when a user taps the ribbon.

7. What is a `StatelessWidget` example? `HomeScreen`, which renders data handed down by its parent.

8. What is a `StatefulWidget` example? `TrainScreen`, which owns its search query and selected category.

9. Difference between `setState` and `notifyListeners`? `setState` updates a particular widget's local state; `notifyListeners` broadcasts shared store changes.

10. What is `BuildContext`? An object describing a widget's location in the tree; it provides access to theme, navigator and dialogs.

11. What does `ListView.builder` do? Builds visible list entries efficiently from a data collection.

12. Why create custom widgets? To reduce duplicated code and keep UI consistent.

13. How does navigation work? `MainShell` switches tabs using `IndexedStack` and `setState`; detail screens use `Navigator.push` / `pop`.

14. What does `http.get` or `_client.get` do? Sends an HTTP GET request to retrieve remote data; `NbaService` owns this logic.

15. What is an API? A defined interface through which our app requests data from an external service.

16. What is JSON? A structured text format used by web APIs and by our storage serialization.

17. Why check `statusCode`? A successful network connection doesn't guarantee a valid response; non-200 statuses signal errors.

18. Why use `try/catch`? To recover from failures (such as offline internet or unsuccessful saving) and show useful feedback instead of crashing.

19. What is `Future`? A Dart object representing an asynchronous result that completes later.

20. Why are `mounted` checks important? The user can close a screen while a Future runs; updating a disposed screen is unsafe.

21. How do sessions survive an app restart? The store encodes records as JSON strings in `shared_preferences` and restores them in `main()`.

22. How does deletion work? Confirm, identify by unique `id`, remove from the correct list, save, and update listeners.

23. What prevents incorrect form input? Validators reject blank titles/descriptions and durations outside 1–180 minutes.

24. What makes the design user-friendly? Consistent controls and colors, labels, progress feedback, readable layout, confirmations and recovery actions.

25. What happens without internet? Training and history still work locally. The Community tab shows error/retry and optional offline sample tips.

26. Is Community a chat feature? No. It is an NBA news and inspiration feed, not a messaging network.

27. Does the ESPN API store workouts? No. ESPN only supplies news. Workouts stay in `shared_preferences` on the device.

28. What is the difference between `toJson` and `fromJson`? `toJson` converts an object into serializable fields; `fromJson` reconstructs an object from those fields.

29. Why a weekly goal range of 1–7? It gives clear achievable targets, and the UI prevents values outside the range.

30. How do you test the API without internet? Inject an `http.Client` into `NbaService` and use `MockClient` to simulate success, server failure, or invalid JSON.

31. What does the timer calculate? It compares the current time against a saved end timestamp, rather than trusting that callbacks fire exactly every second.

32. What is a weakness of your current app? No multi-device synchronization, no login, no proof that an athlete completed a drill; ESPN is a third-party feed that could change. These can be future improvements.

33. What did you learn from your two activities? The routing activity taught multi-page/tab navigation; the API activity taught `http`, `jsonDecode`, `StatefulWidget` and `ListView.builder`. Court IQ combines them with storage and form management.

7. Demonstration flow for the presentation (about 3-5 minutes)

1. **Home** - introduce Court IQ and its target users; point to the floating five-icon ribbon.
2. **Train** - search "shooting" or filter by Shooting; create a short custom drill. Explain form validation.
3. **Workout** - open the custom drill, demonstrate start/pause/reset, then manually log completion. Say that this represents a completed training session entered by the athlete.
4. **Progress** - show count/chart update; tap delete, first Cancel, then Delete. Explain `removeWhere`, saving and listener updates.
5. **Community** - refresh and show *live* data if online; explain `http`, JSON, spinner, retry and fallback. If offline tips appear, be transparent: they're sample content.
6. **Profile** - edit name and weekly goal. Go back to Home; if time allows, restart app to demonstrate persistence.

One-minute speaking script: "Our app is Court IQ, a basketball training companion made with Flutter and Dart. It offers built-in drills, a configurable workout timer, a training log, and weekly progress tracking. The bottom navigation is a reusable custom widget. We use `setState` for local UI changes and `AppStore` with `ChangeNotifier` for shared data. Completed sessions and custom drills are saved locally with `shared_preferences`, so they remain after restarting the app. The Community tab demonstrates API integration by fetching NBA headlines over HTTP and decoding JSON. We validate forms, confirm deletion, and show loading and error states to follow HCI design principles."

8. Quick-fire terminology

Term	Meaning
`Scaffold`	Standard Material screen structure (AppBar, body, etc.).
`AppBar`	Top bar identifying the app/screen.
`SafeArea`	Keeps content away from notches and system UI edges.
`Expanded`	Gives a child available space in a Row/Column.
`ListView`	Scrollable vertical or horizontal content.
`IndexedStack`	Only selected tab is visible while offscreen children keep state.
`Navigator`	Pushes and pops screen routes.
`SnackBar`	Brief message following a user action.
`Form` /validator	Prevents invalid inputs from being submitted.
`ChangeNotifier`	An object that announces changes to listening widgets.
`SharedPreferences`	Simple on-device key-value persistence, not a cloud server.
`jsonEncode`	Converts a Dart map/list to JSON text.
`jsonDecode`	Converts JSON text to Dart objects.
`http.Client.get`	Retrieves data from an HTTP endpoint.
`Timer.periodic`	Schedules repeating callbacks; not guaranteed perfectly precise.

Term	Meaning
`mounted`	Whether a State/BuildContext still belongs to the widget tree.
`dispose()`	Releases resources such as timers and text controllers.
`setState()`	Tells Flutter to rebuild local stateful UI.
`notifyListeners()`	Tells subscribers that shared data changed.

9. Final readiness checklist

- Run `flutter analyze` and `flutter test` on your own computer. The attached code was statically reviewed here but **not actually compiled** in this environment.
- Build/run on your Android emulator, ideally from a simple path `C:\FlutterProjects\CourtIQ`.
- Test every create, edit, read and delete action; verify Cancel leaves data unchanged.
- Restart the app and verify saved records are still there.
- Confirm internet access and live Community headlines immediately before class.
- Submit the **new** project GitHub repository (with generated Android files) on CASTLY.
- Practice explaining **why each widget or service exists**, not just reading code aloud.

Helpful original reference projects

- `https://github.com/halooord/cc_routingActivity` - routing and multiple screens.
- `https://github.com/halooord/cc_Activity` - API fetch, JSON and `ListView.builder`.